Optimization Techniques: Hyperparameter Tuning

(Grid Search • Random Search • Bayesian Optimization)**

0. Introduction (5 minutes)

In machine learning:

- **Model parameters** = learned automatically (e.g., weights in neural networks).
- **Hyperparameters** = **not learned**, must be **set manually** (e.g., learning rate, C in SVM, max_depth in trees).

Goal of hyperparameter tuning:
Find the combination of hyperparameters that gives the **best performance** on validation data.

---

1. Why Hyperparameter Tuning Matters?

Bad hyperparameters →
underfitting
overfitting
slow convergence
bad accuracy

Good hyperparameters →
faster training
better accuracy
stable model
improved generalization

Example:

- In Random Forest, **n_estimators** and **max_depth** drastically change accuracy.
- In Neural Networks, **learning rate** makes the difference between convergence and divergence.

---

2. Grid Search

2.1 What is Grid Search?

Grid search tries **every possible combination** of hyperparameters from a predefined grid.

Example:
SVM

- C = [0.1, 1, 10]
- gamma = [0.01, 0.1]

Grid search combinations = 3 × 2 = **6 total models** to train.

2.2 How Grid Search Works (Step-by-Step)

1. Select hyperparameters
2. Define list of possible values
3. Train model for every combination
4. Evaluate using cross-validation
5. Choose the best combination

2.3 Diagram (explain on board)

```
      C → [0.1, 1, 10]
gamma ↓
0.01        ■    ■    ■
0.1         ■    ■    ■
```

Each ■ = one model trained.

2.4 Advantages

✔ Exhaustive search → guaranteed best from the grid
✔ Easy to implement
✔ Works well with small number of parameters

2.5 Limitations

Very slow
Computationally expensive
Not feasible with many hyperparameters
Searching in regions that don’t matter

---

3. Random Search

3.1 What is Random Search?

Instead of checking every combination, it picks **random combinations** from the search space.

If grid search would test 100 models, random search may test **15–20**, but still find a near-optimal solution.

3.2 Example

Search space:

- learning_rate = [0.0001 → 0.1]
- batch_size = [16, 32, 64, 128]
- dropout = [0–0.5]

Grid search → 4 × 100 × 50 = **20,000 models**
Random search → test only **20–50 models**

3.3 How Random Search Works

1. Define range of values
2. Randomly pick sets of hyperparameters
3. Train and evaluate
4. Track best result

3.4 Diagram (board-friendly)

```
Searching a rectangle region:
Grid:    X X X X X
         X X X X X
Random:  X    X     X
     X       X   X
```

Random search explores wider space.

3.5 Advantages

✔ Much faster
✔ Efficient for high-dimensional spaces
✔ Good results with fewer trials
✔ Can include continuous ranges

3.6 Limitations

❌ Not guaranteed to find the absolute best
❌ Performance depends on number of iterations

---

4. Bayesian Optimization

4.1 What is Bayesian Optimization?

A **smart** optimization technique that uses previous results to decide **where to search next**.

It builds a **probabilistic model** of the function that maps
hyperparameters → performance.

Uses:

- Gaussian Processes
- TPE (Tree-structured Parzen Estimator)

4.2 Key Idea

Instead of blindly trying combinations, it **predicts where the best result might be**.

This makes it ideal when:

- Model training is expensive
- Hyperparameter space is large
- Need best result fast

4.3 Steps of Bayesian Optimization

1. Choose some hyperparameters (randomly at start)
2. Train model & record performance
3. Build a probability model (Gaussian Process)
4. Select next hyperparameters using **acquisition function**
5. Repeat until optimum is found

4.4 Example Scenario

You tune learning rate & batch size.
Bayesian optimization observes:

- lr=0.01 → bad
- lr=0.001 → better
  It automatically **narrows search** near 0.001 region.

4.5 Acquisition Functions

1. **EI – Expected Improvement**
2. **PI – Probability of Improvement**
3. **UCB – Upper Confidence Bound**

They help decide “where to try next”.

4.6 Diagram (explain on board)

Show curve:

- known points
- predicted curve (Gaussian)
- uncertainty area
- next sample chosen in high expected improvement area.

4.7 Advantages

✔ Requires fewer experiments
✔ Very efficient
✔ Works great for expensive models (Deep Learning, XGBoost)
✔ Learns from prior trials

4.8 Limitations

Hard to implement
More memory & computation
Struggles with very high-dimensional spaces

---

5. Summary Table

Method	Speed	Accuracy	Best For
Grid Search	Slow	High (within grid)	Small search spaces
Random Search	Fast	Good	High-dimensional spaces
Bayesian Optimization	Very Fast	Very High	Expensive models, complex spaces

---

6. Practical Example Code Snippets

Grid Search (sklearn)

```
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(model, param_grid, cv=5)
grid.fit(X, y)
```

Random Search

```
from sklearn.model_selection import RandomizedSearchCV
random = RandomizedSearchCV(model, param_dist, n_iter=20, cv=5)
random.fit(X, y)
```

Bayesian Optimization (Optuna)

```
import optuna

def objective(trial):
    lr = trial.suggest_loguniform("lr", 1e-4, 1e-1)
    depth = trial.suggest_int("depth", 3, 12)
    ...
```